Enterprise Multi-Tenancy — Workflow & Architecture

HCM Hybrid Cloud Portal · Current working model (Phase 1 backend + Phase 2 Administration UI, verified)

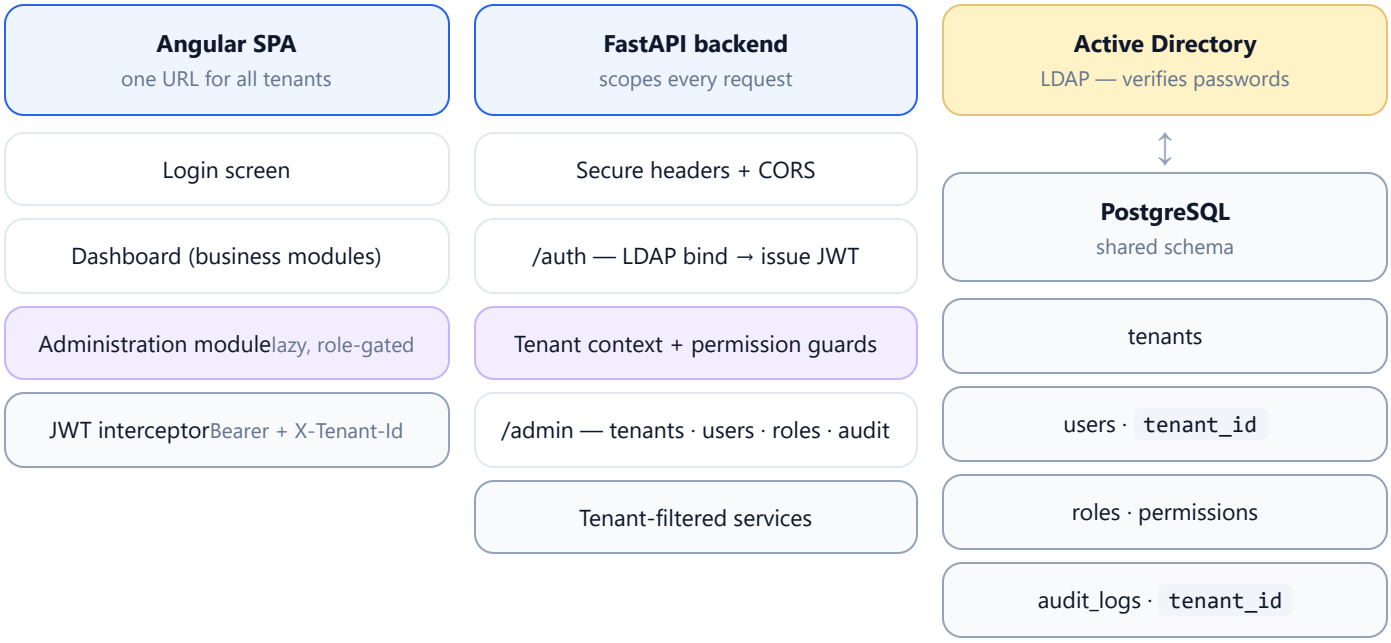
FastAPI + PostgreSQL | Angular 17 | Shared schema + tenant_id | DB-managed RBAC | LDAP authentication | JWT sessions

1 · What this is

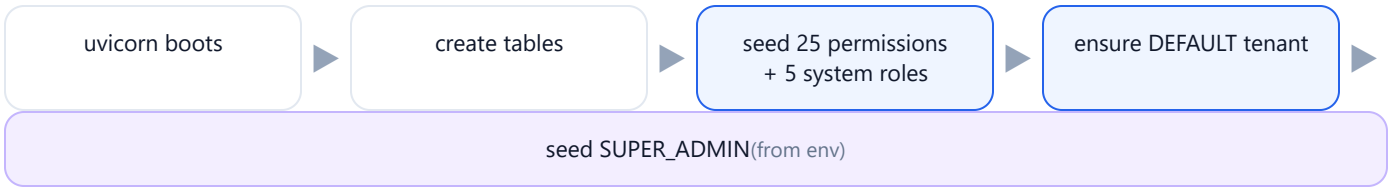
A single deployment that securely serves many isolated **Business Units (tenants)**. Identity is verified against Active Directory over LDAP; the database is the source of truth for which tenant a user belongs to and what they may do. Every API call is scoped to the caller's tenant.

DECISION	CHOICE	WHY
Data isolation	Shared DB + shared schema + `tenant_id` (row-level)	Scales to many BUs; one schema; central enforcement
Authentication	LDAP / Active Directory bind	No passwords stored in the app
Authorization	DB-managed roles & permissions	Admins assign roles; clean audit; no AD-group sprawl
Tenant context	JWT `tenant_id` (+ `X-Tenant-Id` for super-admin switch)	Identity-based, single URL — not subdomain-based
Tenant nesting	None (flat tenants)	Simpler isolation; per requirement

2 · System architecture



3 · Startup & bootstrap (one-time, idempotent)



Re-runs safely on every restart — it only fills gaps and never duplicates. This solves the "first admin" problem: a super admin must exist before any tenant can be onboarded.

4 · Login & token flow

The same path for every role. **LDAP proves who you are; the database decides your tenant and role.**

User	Submits `username` + `password` to `POST /auth/login`
▼	
Backend → AD	LDAP **bind** verifies the password & reads directory attributes (no password is stored)
▼	
Resolve user	Match by `directory_id` → else **claim** an admin-pre-created row by username (binds the real AD GUID) → else *dev:* auto-provision as USER in DEFAULT / *prod:* reject
▼	
Checks	User **ACTIVE**? Tenant **ACTIVE**? — otherwise 403 (audited)
▼	
RBAC	Expand the user's DB roles → effective permission list
▼	
Issue JWT	`{ sub, tenant_id, role, roles[], permissions[], name, email }` · login event written to `audit_logs`
▼	
SPA	Stores token; decodes claims to gate menus & routes

5 · Per-request authorization pipeline

Request Bearer + X-Tenant-Id? ▶ Verify JWT load user ▶ Resolve effective tenant ▶ require_permissions() '*' bypasses

Query filtered by tenant_id

Tenant user
Pinned to their own tenant. A mismatched `X-Tenant-Id` → **403** (cross-tenant blocked).

Super admin
Platform-wide by default; may opt into a tenant via `X-Tenant-Id` (tenant switch).

Verified: cross-tenant access → 403 · privilege escalation (tenant admin granting SUPER_ADMIN) → 403 · tenant-scoped audit reads.

6 · Roles & permissions (RBAC)

ROLE	SCOPE	TENANT_ID	CAN DO
SUPER_ADMIN	Platform	null	Everything (`*`): onboard/activate tenants, create admins, all logs, global settings, switch tenant
TENANT_ADMIN	One tenant	set	Own tenant only: `user:*` `role:*` `audit:read` `tenant_settings:*` — no tenant create, no platform settings
APPROVER	One tenant	set	Read + approve within tenant
USER	One tenant	set	`infra:read` `tenant_settings:read` — dashboards/resources
READ_ONLY	One tenant	set	Read-only within tenant
CUSTOM_ROLE	One tenant	set	Tenant-defined from the permission catalog

7 · Provisioning lifecycle (who creates whom)

1. **SUPER_ADMIN** (seeded at startup) logs in — recognised via `SUPER_ADMIN_USERNAME`.
2. Onboards a **tenant** (Tenant Management): code, name, login type, optional own-LDAP fields.
3. Creates a **TENANT_ADMIN** for that tenant (User Management) — by username; tenant bound automatically.
4. **TENANT_ADMIN** logs in (LDAP) — sees Administration scoped to their tenant only.
5. Creates tenant **USERS** (role bound to their own tenant; cannot grant SUPER_ADMIN).
6. **USER** logs in — dashboard + own-tenant data only; no Administration menu.

8 · Status of the working model

Built & verified

DONE Tenant / RBAC / audit schema + bootstrap seed

DONE LDAP login → JWT with tenant + permissions

DONE Tenant context + permission guards (cross-tenant 403)

DONE Admin APIs: tenants · users · roles · permissions · audit

DONE Angular Administration module (lazy, role-gated, tenant switch)

DONE Screens: Tenant, User, Role mgmt · Permissions · Audit

DONE Backend verified live; frontend build passes

Pending / future

BLOCKER Point `.env` LDAP at a real/test directory (placeholder now)

BLOCKER Set real `SUPER_ADMIN_USERNAME` ; prod provisioning flag

STUB Sub-pages: LDAP Config · Tenant/Platform Settings · Infra · Access Control

TODO Delete endpoints (tenant/user) · role edit UI

PHASE 3 Per-tenant "Own LDAP" login routing

PHASE 3 PostgreSQL Row-Level Security backstop

PHASE 3 Kubernetes namespace isolation + tenant_id on business tables